BASE DE DATOS LIBRERÍA (Triggers, Eventos, Vistas e Índices)

Samuel Esteban Linares Hernández

Servicio Nacional de Aprendizaje, SENA

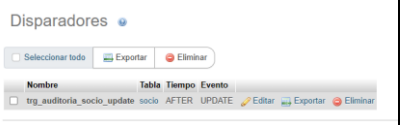
Análisis y Desarrollo de Software

Instructora: Angélica M. Triana

16-03-2025

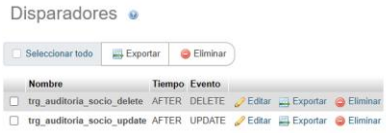
CREATE TRIGGER de auditoria para los update de la tabla socio

```
DELIMITER $$

CREATE TRIGGER trg_auditoria_socio_update
AFTER UPDATE ON socio
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_socio (
        soc_numero,
        accion,
        antiguo_nombre, nuevo_nombre,
        antiguo_apellido, nuevo_apellido,
        antigua_direccion, nueva_direccion,
        antiguo_telefono, nuevo_telefono,
        fecha_modificacion,
        usuario
    ) VALUES (
        OLD.soc_numero,
        'UPDATE',
        OLD.soc_nombre, NEW.soc_nombre,
        OLD.soc_apellido, NEW.soc_apellido,
        OLD.soc_direccion, NEW.soc_direccion,
        OLD.soc_telefono, NEW.soc_telefono,
        NOW(),
        CURRENT_USER()
    );
END $$

DELIMITER ;
```



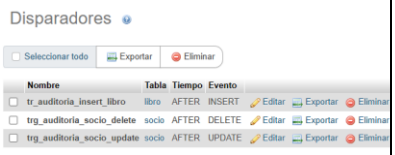
CREATE TRIGGER de auditoria para los delete de la tabla socio

```
DELIMITER $$

CREATE TRIGGER trg_auditoria_socio_delete
AFTER DELETE ON socio
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_socio (
        soc_numero,
        accion,
        antiguo_nombre, nuevo_nombre,
        antiguo_apellido, nuevo_apellido,
        antigua_direccion, nueva_direccion
        antiguo_telefono, nuevo_telefono,
        fecha_modificacion,
        usuario
    ) VALUES (
        OLD.soc_numero,
        'DELETE',
        OLD.soc_nombre, NULL,
        OLD.soc_apellido, NULL,
        OLD.soc_direccion, NULL,
        OLD.soc_telefono, NULL,
        NOW(),
        CURRENT_USER()
    );
END $$

DELIMITER ;
```



Hacer trigger de auditoria para los insert en la tabla libro

```
DELIMITER $$
CREATE TRIGGER tr_auditoria_insert_libro AFTER INSERT ON libro
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_libro (
        aud_lib_isbn,
        aud_lib_titulo,
        usuario,
        aud_fecha_registro
    )
    VALUES (
        NEW.lib_isbn,
        NEW.lib_titulo,
        USER(),
        NOW()
    );
END
$$
DELIMITER ;
```



Hacer trigger de auditoria para los update en la tabla libro

```
DELIMITER $$

CREATE TRIGGER tr_auditoria_update_libro
AFTER UPDATE ON libro
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_libro (
        aud_lib_isbn,
        aud_lib_titulo,
        usuario,
        aud_fecha_registro,
        accion
    )
    VALUES (
        OLD.lib_isbn,
        CONCAT('Cambio de: ', OLD.lib_titulo, ' a: ', NEW.lib_titulo),
        USER(),
        NOW(),
        'UPDATE'
    );
END$$

DELIMITER ;
```

Disparadores

Seleccionar todo | Exportar | Eliminar

	Nombre	Tabla	Tiempo	Evento			
☐	tr_auditoria_insert_libro	libro	AFTER	INSERT	Editar	Exportar	Eliminar
☐	tr_auditoria_update_libro	libro	AFTER	UPDATE	Editar	Exportar	Eliminar
☐	trg_auditoria_socio_delete	socio	AFTER	DELETE	Editar	Exportar	Eliminar
☐	trg_auditoria_socio_update	socio	AFTER	UPDATE	Editar	Exportar	Eliminar

Hacer trigger de auditoria para los delete en la tabla libro (que no tenga dependencias)

```
DELIMITER $$

CREATE TRIGGER tr_auditoria_delete_libro
AFTER DELETE ON libro
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_libro (
        aud_lib_isbn,
        aud_lib_titulo,
        usuario,
        aud_fecha_registro,
        accion
    )
    VALUES (
        OLD.lib_isbn,
        OLD.lib_titulo,
        USER(),
        NOW(),
        'DELETE'
    );
END$$

DELIMITER ;
```

Disparadores

Seleccionar todo | Exportar | Eliminar

	Nombre	Tabla	Tiempo	Evento			
☐	tr_auditoria_delete_libro	libro	AFTER	DELETE	Editar	Exportar	Eliminar
☐	tr_auditoria_insert_libro	libro	AFTER	INSERT	Editar	Exportar	Eliminar
☐	tr_auditoria_update_libro	libro	AFTER	UPDATE	Editar	Exportar	Eliminar
☐	trg_auditoria_socio_delete	socio	AFTER	DELETE	Editar	Exportar	Eliminar
☐	trg_auditoria_socio_update	socio	AFTER	UPDATE	Editar	Exportar	Eliminar

Realizar trigger de auditoria para la tabla autor en donde se registren las actualizaciones

```
DELIMITER $$

CREATE TRIGGER trg_auditoria_autor_update
AFTER UPDATE ON autor
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_autor (
        aud_aut_codigo,
        accion,
        antiguo_apellido, nuevo_apellido,
        antigua_muerte, nueva_muerte,
        fecha_modificacion,
        usuario
    ) VALUES (
        OLD.aut_codigo,
        'UPDATE',
        OLD.aut_apellido, NEW.aut_apellido,
        OLD.aut_muerte, NEW.aut_muerte,
        NOW(),
        CURRENT_USER()
    );
END
$$

DELIMITER ;
```

	Nombre	Tabla	Tiempo	Evento			
☐	tr_auditoria_delete_libro	libro	AFTER	DELETE	Editar	Exportar	Eliminar
☐	tr_auditoria_insert_libro	libro	AFTER	INSERT	Editar	Exportar	Eliminar
☐	tr_auditoria_update_libro	libro	AFTER	UPDATE	Editar	Exportar	Eliminar
☐	trg_auditoria_autor_update	autor	AFTER	UPDATE	Editar	Exportar	Eliminar
☐	trg_auditoria_socio_delete	socio	AFTER	DELETE	Editar	Exportar	Eliminar
☐	trg_auditoria_socio_update	socio	AFTER	UPDATE	Editar	Exportar	Eliminar

Realizar trigger de auditoria para la tabla autor en donde se registren las actualizaciones

```
DROP TRIGGER IF EXISTS trg_auditoria_autor_update;


DELIMITER $$

CREATE TRIGGER trg_auditoria_autor_update
AFTER UPDATE ON autor
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_autor (
        aud_aut_codigo,
        accion,
        antiguo_apellido,
        nuevo_apellido,
        antigua_muerte,
        nueva_muerte,
        fecha_modificacion,
        usuario
    ) VALUES (
        OLD.aut_codigo,
        'UPDATE',
        OLD.aut_apellido,
        NEW.aut_apellido,
        OLD.aut_muerte,
        NEW.aut_muerte,
        NOW(),
        CURRENT_USER()
    );
END$$

DELIMITER ;
```

☐	trg_auditoria_autor_update	autor	AFTER	UPDATE	Editar	Exportar	Elimina

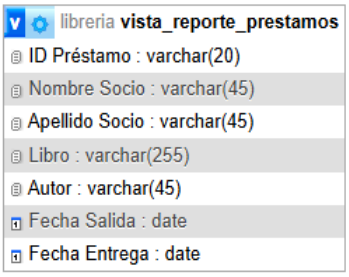
Evento para eliminar préstamos. *Agregar start y ends al evento.		

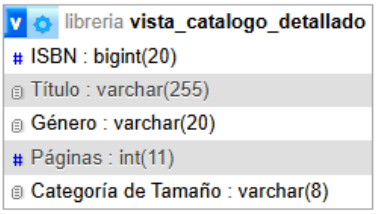
```sql
SET GLOBAL event_scheduler = ON;

DELIMITER $$

CREATE EVENT ev_limpiar_prestamos_antiguos
ON SCHEDULE EVERY 1 MONTH
STARTS '2026-03-16 23:59:59'
ENDS '2027-03-16 23:59:59'
DO
BEGIN
    -- Eliminamos préstamos con fecha de devolución mayor a 365 días
    DELETE FROM prestamo
    WHERE pres_fechaDevolucion < DATE_SUB(NOW(), INTERVAL 1 YEAR);
END$$

DELIMITER ;
```

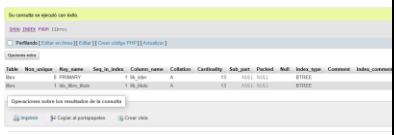
Realizar dos (2) vistas de las consultas que pueden ser más comunes en este caso. (Tipo reporte + de una tabla)

```sql
CREATE VIEW vista_reporte_prestamos AS
SELECT
    p.pres_id AS 'ID Préstamo',
    s.soc_nombre AS 'Nombre Socio',
    s.soc_apellido AS 'Apellido Socio',
    l.lib_titulo AS 'Libro',
    a.aut_apellido AS 'Autor',
    p.pres_fechaPrestamo AS 'Fecha Salida',
    p.pres_fechaDevolucion AS 'Fecha Entrega'
FROM prestamo p
INNER JOIN socio s ON p.soc_copiaNumero = s.soc_numero
INNER JOIN libro l ON p.lib_copiaISBN = l.lib_isbn
INNER JOIN tipoautores ta ON l.lib_isbn = ta.copia_ISBN
INNER JOIN autor a ON ta.copia_autor = a.aut_codigo
WHERE ta.tipoAutor = 'Autor';
```

```sql
CREATE VIEW vista_catalogo_detallado AS
SELECT
    lib_isbn AS 'ISBN',
    lib_titulo AS 'Título',
    lib_genero AS 'Género',
    lib_numeroPaginas AS 'Páginas',
    CASE
        WHEN lib_numeroPaginas > 500 THEN 'Extenso'
        WHEN lib_numeroPaginas BETWEEN 200 AND 500 THEN 'Promedio'
        ELSE 'Corto'
    END AS 'Categoría de Tamaño'
FROM libro;
```





Realizar un index en la tabla libro para la columna lib_titulo.

```sql
CREATE INDEX idx_libro_titulo ON libro (lib_titulo);
```